Iterate over each row in example table

Extract field values for row

Create new account record using these values

```
@Given("the following accounts: $accounts")
public void givenTheFollowingAccounts(ExamplesTable accounts) {
   for(Parameters account : accounts.getRowsAsParameters())
      String owner = account.valueAs("owner", String.class);
      int points = account.valueAs("points", Integer.class);
      int statusPoints = account.valueAs("statusPoints", Integer.class);
      inTheTestDatabase.addAccount(Account.forMember(owner)
                                          .withPointBalance(points)
                                          .withStatusPoints(status));
   }
}
```

An equivalent Cucumber implementation written in Ruby, on the other hand, could use some of Ruby's dynamic language features to write a more concise step implementation:

Pass in the accounts table.

Convert the table data to a list of hash maps, and use this to create corresponding account records.

```
Given /^the following accounts$/ do |accounts|
  Account.create!(accounts.hashes)
end
```

Most tools also let you use more advanced API or language-related features to convert example tables directly into domain objects, which can simplify the automation code considerably. We'll look at some examples of these more advanced features in the sections on each specific tool, later in this chapter.

The other main use of tabular data is to compare actual results to expected results. For example, you might want to check that records in the database have been correctly added or updated, or that you obtain the search results you expect from a multi-criteria search. But writing custom logic to compare the results row by row is time-consuming and error-prone.

Fortunately all the tools make it easy to compare sets of tabular data, either with other tables that you build or by comparing the data tables with lists of hash maps or domain objects. For example, using Cucumber-JVM in Java, you can directly compare a table with a list of domain objects:

Load the relevant accounts from the database.

Compare their field values with the values in the expected table.

```
@Then("^the accounts should be the following:$")
public void the_accounts_should_be_the_following(DataTable expected)
  throws Throwable {
    List<Account> actualAccounts = loadCurrentAccountsFor(fromMember,
                                                          toMember);
    expectedAccounts.diff(actualAccounts);
}
```

When you use the provided table APIs, error reporting is better and troubleshooting easier. For example, when you run this scenario, if the accounts aren't what you expect, the tools will report what data you were expecting and what it actually found.

6.2.6 Implementing example-based scenarios

You saw in chapter 5 how you can summarize a number of related examples into a single scenario using tables of examples, like the following: